UNIVERSITÉ DE RENNES I
U.F.R. DE MATHÉMATIQUES

UNIVERSITÉ DE RENNES 1

Master 2 Mathématiques et applications
Parcours Calcul scientifique et modélisation
U.E. Logiciels Éléments Finis

Solving the transient heat conduction problem with Freefem++

Goal : to solve linear and nonlinear transient heat conduction problems with Freefem++ in 2D.

1 The transient heat conduction problem

Let us consider an isotropic body occupying a domain $\overline{\Omega}$ in space. The mechanism of heat transfer by conduction can be described by the following governing equation

$$\rho\, c_p\, \frac{\partial T}{\partial t}(\mathbf{x},t) - \operatorname{div}\left(k\, \nabla_{\mathbf{x}} T(\mathbf{x},t)\right) = Q(\mathbf{x},t) \qquad \forall \mathbf{x} \in \Omega \quad \forall t \in \mathbb{R}^+ \tag{1}$$

where k is the thermal conductivity of the material in units $[\mathrm{W\,m^{-1}\,K^{-1}}]$, ρ and c_p are respectively the density in units $[\mathrm{Kg\,m^{-3}}]$ and specific heat capacity (heat capacity per unit mass of a material) in units $[\mathrm{J\,Kg^{-1}\,K^{-1}}]$ and Q is the inner heat generation rate per unit volume (heat source density) in units $[\mathrm{W\,m^{-3}}]$. The heat flow is expressed as

$$\mathbf{q}(\mathbf{x},t) = -k\, \nabla_{\mathbf{x}} T(\mathbf{x},t).$$

The boundary conditions on parts of the boundary $\partial\Omega$ can be of the following type :

1. Specified temperature : $T = T_e$ where T_e is a known environmental temperature
2. Specified heat flow : $\mathbf{q}\cdot\mathbf{n} = -\alpha\, q_s$ where q_s is a known heat flow per unit surface area and α is the surface absorption coefficient.
3. Convection boundary conditions : $\mathbf{q}\cdot\mathbf{n} = h(T - T_e)$ where T_e is a known environmental temperature and h is the convection coefficient or heat transfer coefficient (in units $[\mathrm{W\,m^{-2}\,K^{-1}}]$).
4. Radiation : $\mathbf{q}\cdot\mathbf{n} = \sigma\varepsilon(T^4 - T_e^4)$ where σ is the Stefan-Boltzmann constant ($\sigma = 5,670373\,10^{-8}$ $\mathrm{W\,m^{-2}\,K^{-4}}$) and ε is the surface emission coefficient.

The last 3 conditions can be mixed together to give rise to the more general condition

$$-k\frac{\partial T}{\partial n} = \mathbf{q}\cdot\mathbf{n} = h(T - T_e) + \sigma\varepsilon(T^4 - T_e^4) - \alpha q_s.$$

For transient problems it is necessary to specify the temperature field in Ω at the time $t = 0$: $T(\mathbf{x},0) = T_0(\mathbf{x})$ for all $\mathbf{x} \in \Omega$.

The PDE (1) involves derivation of the temperature with respect to time. An usual way of solving (1) with appropriate boundary conditions consists in partially discretizing the equation with respect to time and to solve the resulting equation by the Finite Element Method. Thus, we divide the time domain $[0, t_{\max}]$ into N equal intervals $[t_n, t_{n+1}]$. When the step size $\delta t = T/N$ is sufficiently small, we can approximate the partial derivative with respect to time in the transient heat equation (1) at time t_n with a finite difference formula as

$$\frac{\partial T}{\partial t}(\mathbf{x},t_n) \approx \frac{T(\mathbf{x},t_n) - T(\mathbf{x},t_{n-1})}{\delta t}. \tag{2}$$

Given the initial temperature distribution at time $t = 0$, we compute step by step the temperature at time t_n for all $n \in \{1, \ldots, N\}$ in Ω by solving the PDE

$$\rho c_p \frac{T(\mathbf{x}, t_n)}{\delta t} - \operatorname{div}\left(k \nabla T(\mathbf{x}, t_n)\right) = Q(\mathbf{x}, t_n) + \rho c_p \frac{T(\mathbf{x}, t_{n-1})}{\delta t} \qquad \forall \mathbf{x} \in \Omega. \tag{3}$$

2 A bi-dimensional test problem

Let Ω be the domain corresponding to a slab with a hole as depicted in figure 1. We denote by Γ_1 the outer boundary and by Γ_2 the inner boundary of the domain. We assume that the slab corresponds to the square $[0, 1] \times [0, 1]$ and we denote by R (with $R < 1$) the radius of the hole centered at the origin of the frame.

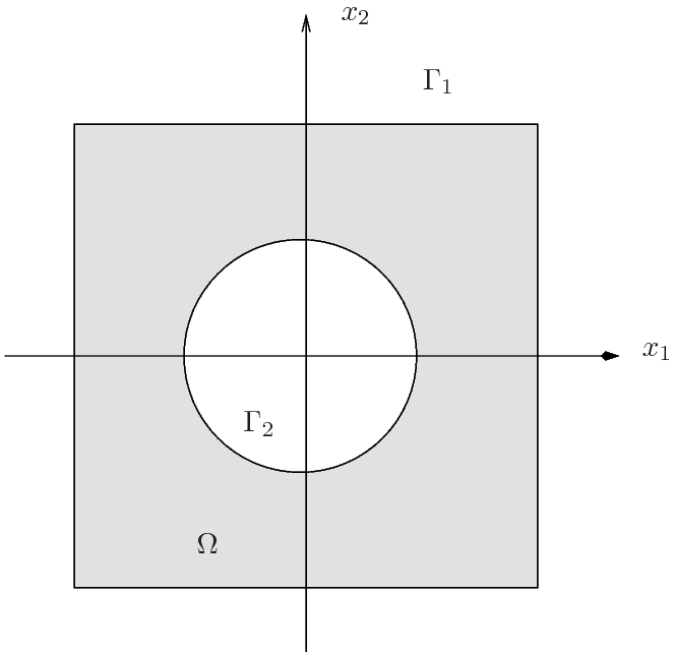


FIGURE 1 – Geometrical domain under consideration.

We look for a regular function $u : (x_1, x_2, t) \in \overline{\Omega} \times [0, t_{\max}] \mapsto u(x_1, x_2, t) \in \mathbb{R}$ solution to the following problem :

$$\left\{ \begin{array}{rcll} \dfrac{\partial u}{\partial t}(x_1, x_2, t) - \Delta u(x_1, x_2, t) & = & f(x_1, x_2, t) & \forall (x_1, x_2) \in \Omega \quad \forall t \in [0, t_{\max}] \\ u(x_1, x_2, t) & = & 0 & \forall (x_1, x_2) \in \Gamma_1 \quad \forall t \in [0, t_{\max}] \\ u(x_1, x_2, t) & = & g(x_1, x_2, t) & \forall (x_1, x_2) \in \Gamma_2 \quad \forall t \in [0, t_{\max}] \\ u(x_1, x_2, 0) & = & u_0(x_1, x_2) & \forall (x_1, x_2) \in \Omega \end{array} \right. \tag{4}$$

where $t_{\max} > 0$ corresponds to the duration where the phenomenon is observed. Problem (4) corresponds to the transient heat equation (1) after adimensionalisation. The Dirichlet boundary conditions correspond to a specified temperature on the boundary.

As we are considering a test problem, we want to know an exact solution for experimental accuracy investigations on the approximation method. The best way to have an exact solution for the test problem (4) is to choose an expression for the solution and to deduce the data f, g and u_0 from this solution. This way of proceeding is much easier than choosing the data and computing the exact solution (when it is possible).

1 Compute f, g and u_0 so that the solution to problem (4) reads $u(x_1, x_2, t) = (x_1 - x_1^3)(x_2 - x_2^3)\,\mathrm{e}^{-t}$.

Partial discretization with respect to time

We introduce a uniform subdivision $\sigma = (t_n)_{n\in\{0,\dots,N\}}$ of the time interval $[0, t_{\max}]$ with time step denoted δt and we consider the following sequence of PDE problems deduced from (4) and indexed by $n \in \{1, \dots, N\}$: find a regular function $u_n : (x_1, x_2) \in \Omega \mapsto u_n(x_1, x_2) \in \mathbb{R}$ such that

$$\left\{\begin{array}{rcll} \dfrac{u_n(x_1,x_2)}{\delta t} - \Delta u_n(x_1,x_2) &=& \dfrac{u_{n-1}(x_1,x_2)}{\delta t} + f(x_1,x_2,t_n) & \forall (x_1,x_2) \in \Omega \\ u_n(x_1,x_2) &=& 0 & \forall (x_1,x_2) \in \Gamma_1 \\ u_n(x_1,x_2) &=& g_n(x_1,x_2) & \forall (x_1,x_2) \in \Gamma_2 \end{array}\right. \tag{5}$$

where $g_n : (x_1, x_2) \in \Gamma_2 \mapsto g(x_1, x_2, t_n)$.

2 Show that the weak formulation for problem (5) reads : find $u_n \in \mathbb{H}^1(\Omega)$ with $u_n = g_n$ on Γ_2 and $u_n = 0$ on Γ_1 such that for all $v \in \mathbb{H}^1_0(\Omega)$

$$\frac{1}{\delta t}\int_\Omega u_n(x)\, v(x)\,\mathrm{d}x + \int_\Omega \nabla u_n(x)\cdot\nabla v(x)\,\mathrm{d}x = \frac{1}{\delta t}\int_\Omega u_{n-1}(x)\, v(x)\,\mathrm{d}x + \int_\Omega f(x)\, v(x)\,\mathrm{d}x. \tag{6}$$

The following FREEFEM++ program solves the transient heat problem (4) for $t_{\max} = 1$ through the computation of the sequence $(u_n)_{n=1,\dots,N}$ of solutions to problems (5) for $N = 200$.

```
// ---- Geometric domain and mesh ----
real R=0.5;
real Lx=1, Ly=1;
border Gamma1(t=-1,1){x=Lx*t;y=-Ly;label=1;};
border Gamma2(t=-1,1){x=Lx;y=Ly*t;label=1;};
border Gamma3(t=-1,1){x=-Lx*t;y=Ly;label=1;};
border Gamma4(t=-1,1){x=-Lx;y=-Ly*t;label=1;};
border Gamma5(t=0,2*pi){x=R*cos(t);y=R*sin(t);label=2;};
int n=20;
mesh Th=buildmesh(Gamma1(n)+ Gamma2(n)+ Gamma3(n)+ Gamma4(n)+ Gamma5(-2*n));
plot(Th,wait=true,ps="maillage_1.eps");
// ---- Data ----
func f=-x*y*(x^2*y^2+5*(x^2+y^2)-11); // source term in Omega
func g=(x-x^3)*(y-y^3);  // source term on  Gamma5
func u0=(x-x^3)*(y-y^3); // initial data
// ---- Finite element space and weak formulation -----
int N=200; // number of time discretization nodes
real T=1, dt=T/N; // time duration and time step
fespace Vh(Th,P2);
Vh u,v,uold,uex;
macro grad(u) [dx(u),dy(u)] //end of  macro
u=u0; // initialisation
for (real t=0;t<T;t+=dt){ // loop over the time discretization nodes
          uold=u;
          solve chaleur(u,v)=int2d(Th)(u*v/dt + grad(u)'*grad(v))
                      -int2d(Th)(f*v*exp(-t)+uold*v/dt)
                      +on(1,u=0)+on(2,u=g*exp(-t)); // the variational problem is solved
}
// ---- Draw the solutions and compute the error ----
plot(u);
uex=(x-x^3)*(y-y^3)*exp(-T);
Vh err = abs(uex-u);
real L2err=sqrt( int2d(Th)((u-uex)^2)/ int2d(Th)(uex^2));
cout << "Erreur L2 relative (%)= " << 100*L2err << endl;
plot(uex);
plot(err);
```

The solution at time $t = 1$ is depicted in figure 2. The relative error expressed in the $\mathbb{L}^2(\Omega)$ norm at time $t = 1$ is 0.5%.

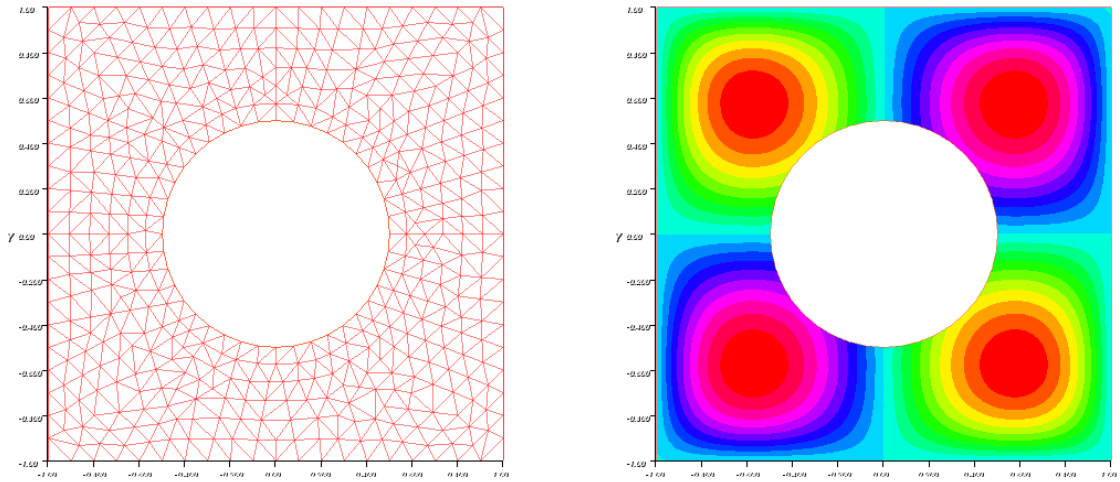


FIGURE 2 – Mesh and temperature distribution in Ω at time $t = 1$.

On the incidence of symmetry of the domain and data on the solution

We can see on the previous example that the data have nice symmetry properties : for all $(x_1, x_2) \in \Omega$ and for all $t \in [0, T]$, we have

$$f(-x_1, x_2, t) = -f(x_1, x_2, t) \quad \text{and} \quad f(x_1, -x_2, t) = -f(x_1, x_2, t),$$
$$u_0(-x_1, x_2) = -u_0(x_1, x_2) \quad \text{and} \quad u_0(x_1, -x_2) = -u_0(x_1, x_2).$$

Moreover, the domain Ω is also symmetric with respect to the x_1 and x_2 axes and for all $(x_1, x_2) \in \Gamma_2$ and for all $t \in [0, T]$, we have

$$g(-x_1, x_2, t) = -g(x_1, x_2, t) \quad \text{and} \quad g(x_1, -x_2, t) = -g(x_1, x_2, t).$$

As a result, the solution itself exhibits symmetries, see figure 2.

3 a) Show that under these conditions of symmetry, problem (4) can be solve over a quarter of the domain Ω, say in the domain $\widehat{\Omega} = \{(x_1, x_2) \in \Omega \ ; \ x_1 \geq 0, x_2 \geq 0\}$ and the solution over the whole domain Ω deduced from the solution over $\widehat{\Omega}$. What are the boundary conditions to be set on the axes $x_1 = 0$ and $x_2 = 0$?

b) Modify the above given FREEFEM++ program to solve problem (4) in $\widehat{\Omega}$.

4 (Optional) Actually, the computation domain exhibits as well a symmetry with respect to the first bisector (the line with equation $x_1 = x_2$) and it is easy to check that

$$\forall t \in [0, T] \quad \forall (x_1, x_2) \in \Omega \quad f(x_2, x_1, t) = f(x_1, x_2, t) \quad \text{and} \quad u_0(x_2, x_1) = u_0(x_1, x_2)$$
$$\forall t \in [0, T] \quad \forall (x_1, x_2) \in \Gamma_2 \quad g(x_2, x_1, t) = g(x_1, x_2, t).$$

As a consequence, the solution itself exhibits the same symmetry property. Modify your FREEFEM++ program to solve problem (4) in the one eighth of the initial domain Ω.

A radiant floor heating system

We consider a radiant floor heating system where a concrete slab with a thickness of 0.2 m is located between two rooms in which the temperatures are different. The concrete slab is crossed by pipes with a diameter of 0.04 m and spaced out by 0.4 m in which hot water flows. Given the symmetries of the

domain (see the discussion above), we only take into account a portion of the floor, see figure 3. We impose symmetry boundary conditions on the left and right edges of the computational cell. Heating is provided by the circulation of water at 50 ° C in the pipe. We assume that the water flows quickly enough so that the wall of the pipe remains at 50 ° C. We also assume free convection boundary condition ($h_c = 20\ \mathrm{W\,m^{-2}\,K^{-1}}$) with the ambient air at 21 ° C in the room above and with the ambient air at 5 °C in the room below. The thermal conductivity of concrete is $1.75\ \mathrm{W\,m^{-1}\,K^{-1}}$. The density of normal concrete is $2400\ \mathrm{kg\,m^{-3}}$ and its specific heat capacity is $750\mathrm{J\,kg^{-1}K^{-1}}$. At the initial time, the concrete slab is at a uniform temperature of 15 ° C. We are interested in the study of the variation of the temperature in the concrete slab in the transient regime before the system has reached a steady state.

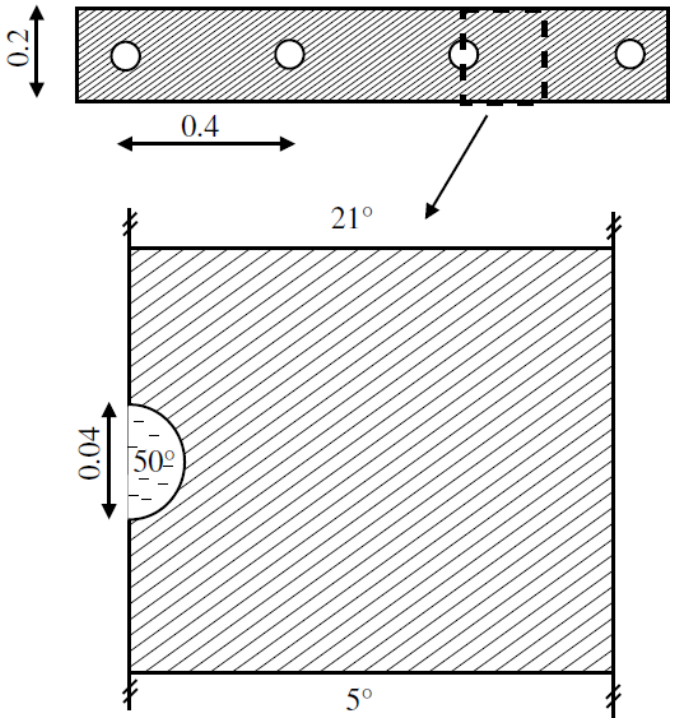


FIGURE 3 – A radiant floor heating system (L. Champaney, ENS Cachan).

5 a) Give the PDE problem to be solved and then give its weak formulation.

b) Modify the above given FREEFEM++ program to compute the temperature distribution in the concrete slab at different time during the transient regime. (Mind the units !)

c) Compute the temperature distribution in the concrete slab once the steady state regime is reached.

3 Nonlinear heat transfer in a thin plate

This problem is proposed in the Matlab documentation as an example of how to use the Partial Differential Equation Toolbox. [1]

3.1 The steady state problem

We consider a plate with planar dimensions one meter by one meter and 1 cm thick, see figure 4. Because the plate is relatively thin compared with the planar dimensions, the temperature can be assumed constant in the thickness direction ; the resulting problem is 2D.

We denote by Ω the open set occupied by the plate. The temperature of the plate is fixed along the bottom edge (denoted Γ_1 in the following) and no heat is transferred from the other three edges (denoted Γ_2 in

1. `http://www.mathworks.fr/help/pde/ug/nonlinear-heat-transfer-in-a-thin-plate.html`

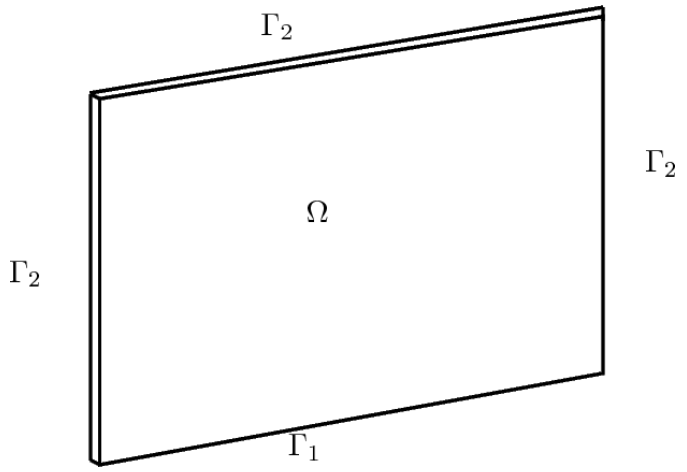


FIGURE 4 – A thin plate.

the following), i.e. they are insulated. Convection and radiation heat transfer are assumed to take place between the two faces of the plate and a specified ambient temperature. We are interested by a steady state analysis and look for the final temperature in the plate after it has reached an equilibrium state.
The amount of heat transferred from each plate face per unit area due to convection is defined as

$$Q_c = h_c(T - T_a)$$

where T_a is the ambient temperature, T is the temperature at a particular location on the plate surface, and h_c is a specified convection coefficient. The amount of heat transferred from each plate face per unit area due to radiation is defined as

$$Q_r = \epsilon\sigma(T^4 - T_a^4)$$

where ϵ is the emissivity of the face and σ is the Stefan-Boltzmann constant. Because the heat transferred due to radiation is proportional to the fourth power of the surface temperature, the problem is nonlinear.
The plate is composed of copper which has the following properties

— thermal conductivity in W/(mK) : $k = 400$
— Convection coefficient in W/(m^{2}K) : $h = 1$
— emissivity of the plate surface $\varepsilon = 0.5$

The ambient temperature is assumed to be 300 degrees-Kelvin.
The PDE describing the temperature in this thin plate is then

$$-ke_z\Delta T + 2Q_c + 2Q_r = 0$$

where e_z is the plate thickness, and the factors of two in front of Q_c and Q_r account for the heat transfer from both plate faces. Thus the PDE boundary problem can be reformulated as : find T such that

$$\left\{\begin{array}{rcll} -ke_z\Delta T + 2h_c(T - T_a) + 2\epsilon\sigma(T^4 - T_a^4) & = & 0 & \text{in } \Omega \\ T & = & T_b & \text{on } \Gamma_1 \\ \dfrac{\partial T}{\partial n} & = & 0 & \text{on } \Gamma_2 \end{array}\right. \tag{7}$$

Since problem (7) involves a nonlinear PDE, a special treatment has to be done before solving it by the FEM. First we introduce as new unknown $u = T - T_a$. We have $\Delta T = \Delta u$ and

$$T^4 - T_a^4 = (T - T_a)(T + T_a)(T^2 + T_a^2) = u\underbrace{(u + 2T_a)((u + T_a)^2 + T_a^2)}_{= g(u)}$$

and the boundary conditions read : $\frac{\partial u}{\partial n} = 0$ and $u = T_b - T_a$.

We consider the following sequence of PDE problems indexed by $i \in \mathbb{N}$: for $i = 0$

$$\left\{ \begin{array}{rcll} -ke_z \Delta u_0 + 2h_c u_0 & = & 0 & \text{in } \Omega \\ u_0 & = & T_b - T_a & \text{on } \Gamma_1 \\ \dfrac{\partial u_0}{\partial n} & = & 0 & \text{on } \Gamma_2 \end{array} \right. \tag{8}$$

and for $i \in \mathbb{N}^*$,

$$\left\{ \begin{array}{rcll} -ke_z \Delta u_i + 2h_c u_i + 2\epsilon\sigma\, u_i g(u_{i-1}) & = & 0 & \text{in } \Omega \\ u_i & = & T_b - T_a & \text{on } \Gamma_1 \\ \dfrac{\partial u_i}{\partial n} & = & 0 & \text{on } \Gamma_2 \end{array} \right. \tag{9}$$

It can be shown that the sequence $(u_i)_{i\in\mathbb{N}}$ converges toward u when i tends to infinity.

6 Write the weak formulation of problems (8) for the unknown u_0 and (9) for the unknown u_i for $i \in \mathbb{N}^*$ fixed.

The following program solve the sequence of problems defined in (8)– (9) under FREEFEM++.

```
real L=1; // size of the plate in m
int N=30; // nb of discretization nodes per square edge
mesh Th=square(N,N,[L*x,L*y]);
plot(Th);
real k = 400; // thermal conductivity of copper, W/(m-K)
real tz = .01; // plate thickness in meters
real stefanBoltz = 5.670373e-8; // Stefan-Boltzmann constant, W/(m^2-K^4)
real h = 1; // Convection coefficient, W/(m^2-K)
real Ta = 300; //  ambient temperature in degree-Kelvin
real Tb = 1000; // temperature at the bottom edge
real epsilon = .5; // emissivity of the plate surface
fespace Vh(Th,P2);
Vh u,uold,v,g=0;
macro grad(u)[dx(u),dy(u)]//fin macro
problem thermnl(u,v)=int2d(Th)(k*tz*grad(u)'*grad(v)+2*h*u*v)+int2d(Th)(g*u*v)
                                        +on(1,u=Tb-Ta);
thermnl; // u is initialized by solving the linear problem (g=0)
int iterx=200, i=0;
real err=1, normu=0;
while (err>1E-6 & i<iterx){
        i=i+1; // iteration counter
        uold=u;
    g=2*stefanBoltz*epsilon*(uold+2*Ta)*((uold+Ta)^2+Ta^2);
    thermnl; // problem is solved
        normu=sqrt(int2d(Th)(abs(u)^2)); // norm of the solution
        cout << "Norm of the solution at iterate "<< i <<" = "<< normu << endl;
    err=sqrt(int2d(Th)(abs(u-uold)^2))/normu; // difference with the last iterate
}
cout << "Number of iterations = " << i << endl;
Vh T=u+Ta; // Temperature in Kelvin
plot(T,ps="main2_sol.eps",fill=true,nbiso=50);
```

Iterations are stopped when the difference between 2 successive iterates is not anymore significant. The difference is evaluated here by the $\mathbb{L}^2$ norm and compared to a relative tolerance value of 10^{-6}. Computations stop after 22 Iterations.

Additionally, we can obtain the temperature value at the top edge of the plate and draw the temperature distribution along the plate as a function of the y coordinate by the following instructions under FREEFEM++.

```
cout << "Temperature at the top edge = " << T(L/2.,L) << endl;
normu=sqrt(int2d(Th)(abs(u+Ta)^2));
cout << "Norm of the solution = "<< normu << endl;
// Draw the temperature as a function of the Y-coordinate
ofstream ff("temp_y.txt");
real dy=L/N;
real[int] tabt(N+1), taby(N+1);
i=0;
for (real y=0;y<=L;y+=dy){
        taby[i]=y;
    tabt[i]=T(L/2.,y);
        ff<<taby[i] << " " << tabt[i] <<endl;
    i=i+1;
}
// To generate the draw with gnuplot directly form Freefem++
// This works only under Linux when gnuplot is installed.
// Otherwise, generate the plot from file "temp_y.txt"
{
ofstream gnu("plot.gp");
for (i=0;i<N;i++){
        gnu << taby[i]<< " "<<tabt[i]<< endl;
}
}
exec("echo 'plot \"plot.gp\"\
pause 10 \
set term postscript \
set output \"gnuplot.eps\"\
replot \
quit '| gnuplot");
```

3.2 The transient problem

In a steady state analysis we are interested in the final temperature in the plate after it has reached an equilibrium state. In a transient analysis we are interested in the temperature in the plate as a function of time. One question that can be answered by this transient analysis is how long does it take for the plate to reach an equilibrium temperature.
The transient heat equation reads

$$\rho C_p e_z \frac{\partial T}{\partial t} - k e_z \Delta T + 2h_c(T - T_a) + 2\epsilon\sigma(T^4 - T_a^4) = 0$$

where ρ is the material density ($\rho = 8960$ kgm^{-3} for copper), C_p is the specific heat ($C_p = 386$Jkg^{-1}K^{-1} for copper) and the other quantities are as defined in the previous section.

7 Modify the previous FREEFEM++ program to solve this transient problem and show that after around 5000 seconds, the transient solution has reached the steady state values (the temperatures from the two solutions at the top edge of the plate agree to within one percent).

Solutions

1 For $u(x_1,x_2,t)=(x_1-x_1^3)(x_2-x_2^3)\,\mathrm{e}^{-t}$, we obtain

$$\frac{\partial u}{\partial t}(x_1,x_2,t)-\Delta u(x_1,x_2,t)=-x_1x_2(x_1^2x_2^2+5(x_1^2+x_2^2)-11)\,\mathrm{e}^{-t}.$$

Therefore $f:(x_1,x_2,t)\mapsto -x_1x_2(x_1^2x_2^2+5(x_1^2+x_2^2)-11)\,\mathrm{e}^{-t}$. We also have $u(x_1,x_2,0)=(x_1-x_1^3)(x_2-x_2^3)$ so that $u_0:(x_1,x_2)\mapsto(x_1-x_1^3)(x_2-x_2^3)$.

2 Let v denote a "test function" belonging to the Sobolev space $\mathbb{H}_0^1(\Omega)$ and assume that the solution u_n to problem (5) belongs to $\mathbb{H}^2(\Omega)$. Then, multiplying each side of the PDE in (5) by v and integrating over Ω yields

$$\int_\Omega \frac{u_n(x)}{\delta t}\,v(x)\,\mathrm{d}x-\int_\Omega \Delta u_n(x)\,v(x)\,\mathrm{d}x=\int_\Omega \frac{u_{n-1}(x)}{\delta t}\,v(x)\,\mathrm{d}x+\int_\Omega f(x_1,x_2,t_n)\,v(x)\,\mathrm{d}x.$$

We then use Green's formula to express the second integral term in the previous relation as

$$\int_\Omega \Delta u_n(x)\,v(x)\,\mathrm{d}x=-\int_\Omega \nabla u_n(x)\cdot\nabla v(x)\,\mathrm{d}x+\oint_{\Gamma_1\cup\Gamma_2}\frac{\partial u_n}{\partial \nu}(x)\,v(x)\,\mathrm{d}\gamma$$

where ν denotes the outward unit normal to the boundary Γ_1 or Γ_2. Since $v\in\mathbb{H}_0^1(\Omega)$, the integral over $\Gamma_1\cup\Gamma_2$ is zero. Finally, we can conclude that the weak formulation for problem (5) reads : find $u_n\in\mathbb{H}^1(\Omega)$ with $u_n=g_n$ on Γ_2 and $u_n=0$ on Γ_1 such that for all $v\in\mathbb{H}_0^1(\Omega)$

$$\frac{1}{\delta t}\int_\Omega u_n(x)\,v(x)\,\mathrm{d}x+\int_\Omega \nabla u_n(x)\cdot\nabla v(x)\,\mathrm{d}x=\frac{1}{\delta t}\int_\Omega u_{n-1}(x)\,v(x)\,\mathrm{d}x+\int_\Omega f(x)\,v(x)\,\mathrm{d}x.$$

3 a) One can see that all the data for the problem are odd functions with respect to the first variable x_1. Since the domain Ω admits the x_2 axis as axis of symmetry, this implies that the solution u_n will also be an odd function with respect to the first variable x_1. Namely

$$\forall x_1\in\mathbb{R}^+\quad\forall x_2\in\mathbb{R}\qquad u_n(-x_1,x_2)=-u_n(x_1,x_2).$$

Therefore, u_n cancels for $x_1=0$. As a consequence, we impose the Dirichlet boundary condition $u_n=0$ on the border $x_2=0$ of the quarter domain $\widehat{\Omega}$.
Similarly, one can see that all the data for the problem are odd functions with respect to the second variable x_2. Since the domain Ω admits the x_1 axis as axis of symmetry, this implies that the solution will also be an odd function with respect to the first variable x_2. Therefore, u_n cancels for $x_2=0$. As a consequence, we also impose the Dirichlet boundary condition $u_n=0$ on the border $x_1=0$ of the quarter domain $\widehat{\Omega}$.
Finally, instead of looking for u_n solution to problem (5), we can look for $\widehat{u}_n:(x_1,x_2)\in\widehat{\Omega}\mapsto\widehat{u}_n(x_1,x_2)\in\mathbb{R}$ such that

$$\left\{\begin{array}{rcll}\dfrac{\widehat{u}_n(x_1,x_2)}{\delta t}-\Delta\widehat{u}_n(x_1,x_2) &=& \dfrac{\widehat{u}_{n-1}(x_1,x_2)}{\delta t}+f(x_1,x_2,t_n) & \forall(x_1,x_2)\in\widehat{\Omega}\\ \widehat{u}_n(x_1,x_2) &=& 0 & \forall(x_1,x_2)\in\widehat{\Gamma}_1\cup\widehat{\Gamma}_3\cup\widehat{\Gamma}_4\\ \widehat{u}_n(x_1,x_2) &=& g_n(x_1,x_2) & \forall(x_1,x_2)\in\widehat{\Gamma}_2\end{array}\right.\qquad(10)$$

where we have denoted $\widehat{\Gamma}_i=\Gamma_i\cap\overline{\widehat{\Omega}}$ for $i=1,2$ and $\widehat{\Gamma_3}=\{(x_1,x_2)\in\overline{\widehat{\Omega}}\mid x_1=0\}$, $\widehat{\Gamma_4}=\{(x_1,x_2)\in\overline{\widehat{\Omega}}\mid x_2=0\}$.
The weak formulation for problem (10) is obtained in a very similar way as the weak formulation for problem (5). It reads : find $\widehat{u}_n\in\mathbb{H}^1(\widehat{\Omega})$ with $\widehat{u}_n=g_n$ on $\widehat{\Gamma}_2$ and $u_n=0$ on $\widehat{\Gamma}_1$, $\widehat{\Gamma}_3$ and $\widehat{\Gamma}_4$ such that for all $\widehat{v}\in\mathbb{H}_0^1(\widehat{\Omega})$

$$\frac{1}{\delta t}\int_{\widehat{\Omega}}\widehat{u}_n(x)\,\widehat{v}(x)\,\mathrm{d}x+\int_{\widehat{\Omega}}\nabla\widehat{u}_n(x)\cdot\nabla\widehat{v}(x)\,\mathrm{d}x=\frac{1}{\delta t}\int_{\widehat{\Omega}}\widehat{u}_{n-1}(x)\,\widehat{v}(x)\,\mathrm{d}x+\int_{\widehat{\Omega}}f(x)\,\widehat{v}(x)\,\mathrm{d}x.$$

b) The following FREEFEM++ program solves problem (10).

```
real R=0.5;
real Lx=1, Ly=Lx;
border Gamma1(t=R,Lx){x=t;y=0;label=1;};
border Gamma2(t=0,Ly){x=Lx;y=t;label=1;};
border Gamma3(t=R,sqrt(2)*Lx){x=t/sqrt(2);y=t/sqrt(2);label=3;};
border Gamma4(t=0,pi/4){x=R*cos(t);y=R*sin(t);label=2;};

int n=20;
mesh Th=buildmesh(Gamma1(n)+ Gamma2(n)+ Gamma3(-n)+ Gamma4(-2*n));
plot(Th,wait=true,ps="maillage_1.eps");

func f=-x*y*(x^2*y^2+5*(x^2+y^2)-11); // terme source dans Omega
func g=(x-x^3)*(y-y^3);  // terme source sur Gamma5
func u0=(x-x^3)*(y-y^3); // donnée initiale

int N=200; // nombre de noeuds de discrétisation en temps
real T=1, dt=T/N; // longueur de l'intervalle en temps et pas de temps

fespace Vh(Th,P2);
Vh u,v,uold,uex;
macro grad(u) [dx(u),dy(u)] //fin macro


ofstream ff("res_main1.txt");
u=u0; // initialisation
for (real t=0;t<T;t+=dt){
uold=u;
solve chaleur(u,v)=int2d(Th)(u*v/dt + grad(u)'*grad(v))
                    -int2d(Th)(f*v*exp(-t)+uold*v/dt)
                    +on(1,u=0)+on(2,u=g*exp(-t)); // on résout le problème variationnel
    ff<<u(R+(Lx-R)/2,R+(Lx-R)/2)<<endl;
}
plot(u,wait=true,fill=true,ps="sol_quart.eps");
uex=(x-x^3)*(y-y^3)*exp(-T);
Vh err = abs(uex-u);
real L2err=sqrt( int2d(Th)((u-uex)^2)/ int2d(Th)(uex^2));
cout << "Erreur L2 relative (%)= " << 100*L2err << endl;
plot(uex,wait=true);
plot(err,wait=true);
```

4 Due to the symmetry of the domain and data mentioned in the text, the solution u_n at time t_n satisfies

$$u_n(x_2, x_1) = u_n(x_1, x_2). \tag{11}$$

Let $\widetilde{\Gamma}_5$ denote the part of the boundary of the one eighth of the initial domain Ω corresponding to the first bisector (the line with equation $x_1 = x_2$). On $\widetilde{\Gamma}_5$, we have $u_n(x_1, x_1) = u_n(x_1, x_1)$ and this relation does not provide any information on the boundary condition to be imposed. The unit normal to $\widetilde{\Gamma}_5$ is $\pm\frac{1}{\sqrt{2}}\begin{pmatrix}1 & -1\end{pmatrix}^\top$ so that the normal derivative of u_n along $\widetilde{\Gamma}_5$ reads

$$\frac{\partial u_n}{\partial n}(x_1, x_2) = \pm\frac{1}{\sqrt{2}}\left(\frac{\partial u_n}{\partial x_1}(x_1, x_2) - \frac{\partial u_n}{\partial x_2}(x_1, x_2)\right).$$

From (11), we deduce that

$$\forall (x_1, x_2) \in \widetilde{\Gamma}_5 \qquad \frac{\partial u_n}{\partial n}(x_1, x_2) = 0.$$

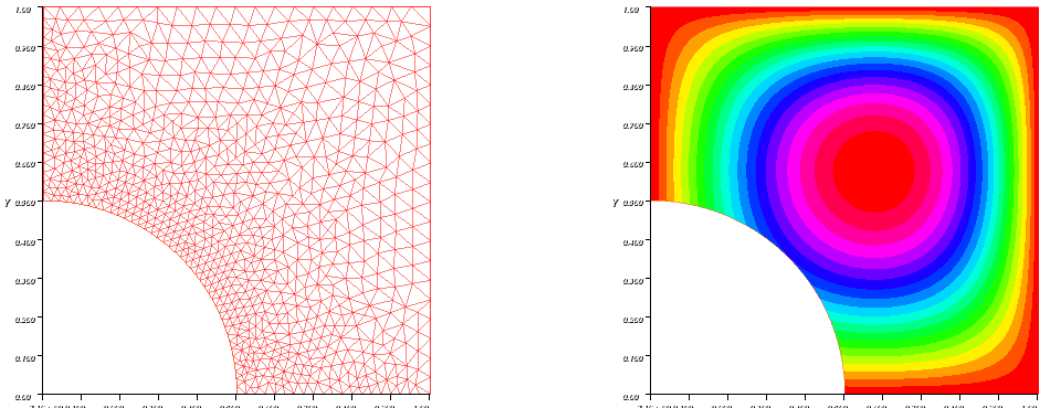


FIGURE 5 – Mesh and temperature distribution in Ω at time $t = 1$.

Let $\widetilde{\Omega} = \{(x_1, x_2) \in \Omega \ ; \ x_1 > 0, x_2 > 0, x_1 < x_2\}$ be the one eighth of the initial domain Ω considered for the simulation. We deduce from (5) and from the above discussion the following boundary problem for u_n : find a regular function $u_n : (x_1, x_2) \in \widetilde{\Omega} \mapsto u_n(x_1, x_2) \in \mathbb{R}$ such that

$$\left\{ \begin{array}{rcll} \dfrac{u_n(x_1,x_2)}{\delta t} - \Delta u_n(x_1,x_2) & = & \dfrac{u_{n-1}(x_1,x_2)}{\delta t} + f(x_1,x_2,t_n) & \forall (x_1,x_2) \in \widetilde{\Omega} \\ u_n(x_1,x_2) & = & 0 & \forall (x_1,x_2) \in \widetilde{\Gamma}_i, i = 1,3,4 \\ u_n(x_1,x_2) & = & g_n(x_1,x_2) & \forall (x_1,x_2) \in \widetilde{\Gamma}_2 \\ \dfrac{\partial u_n}{\partial n}(x_1,x_2) & = & 0 & \forall (x_1,x_2) \in \widetilde{\Gamma}_5 \end{array} \right. \tag{12}$$

where we have denoted $\widetilde{\Gamma}_i = \Gamma_i \cap \overline{\widetilde{\Omega}}$ for $i = 1, 2$ and $\widetilde{\Gamma_3} = \{(x_1, x_2) \in \overline{\widetilde{\Omega}} \mid x_1 = 0\}$, $\widetilde{\Gamma_4} = \{(x_1, x_2) \in \overline{\widetilde{\Omega}} \mid x_2 = 0\}$. and $\widetilde{\Gamma_5} = \{(x_1, x_2) \in \overline{\widetilde{\Omega}} \mid x_2 = x_1\}$.
The weak formulation for problem (12) reads : find $u_n \in \mathbb{H}^1(\widetilde{\Omega})$ with $u_n = g_n$ on Γ_2 and $u_n = 0$ on Γ_1 such that for all $v \in V$

$$\frac{1}{\delta t} \int_{\widetilde{\Omega}} u_n(x)\, v(x)\, \mathrm{d}x + \int_{\widetilde{\Omega}} \nabla u_n(x) \cdot \nabla v(x)\, \mathrm{d}x = \frac{1}{\delta t} \int_{\widetilde{\Omega}} u_{n-1}(x)\, v(x)\, \mathrm{d}x + \int_{\widetilde{\Omega}} f(x)\, v(x)\, \mathrm{d}x. \tag{13}$$

where $V = \{v \in \mathbb{H}^1(\widetilde{\Omega}) \ ; \ v = 0 \text{ on } \Gamma_i, i = 1, 2, 3, 4\}$.

The following FREEFEM++ program solves problem (4) in the one eighth of the initial domain Ω.

```
real R=0.5;
real Lx=1, Ly=Lx;
border Gamma1(t=R,Lx){x=t;y=0;label=1;};
border Gamma2(t=0,Ly){x=Lx;y=t;label=1;};
border Gamma3(t=R,sqrt(2)*Lx){x=t/sqrt(2);y=t/sqrt(2);label=3;};
border Gamma4(t=0,pi/4){x=R*cos(t);y=R*sin(t);label=2;};

int n=20;
mesh Th=buildmesh(Gamma1(n)+ Gamma2(n)+ Gamma3(-n)+ Gamma4(-2*n));
plot(Th,wait=true,ps="maillage_1.eps");

func f=-x*y*(x^2*y^2+5*(x^2+y^2)-11); // terme source dans Omega
func g=(x-x^3)*(y-y^3);  // terme source sur Gamma5
func u0=(x-x^3)*(y-y^3); // donnée initiale
```

```
int N=200; // nombre de noeuds de discrétisation en temps
real T=1, dt=T/N; // longueur de l'intervalle en temps et pas de temps

fespace Vh(Th,P2);
Vh u,v,uold,uex;
macro grad(u) [dx(u),dy(u)] //fin macro

ofstream ff("res_main1.txt");
u=u0; // initialisation
for (real t=0;t<T;t+=dt){
uold=u;
solve chaleur(u,v)=int2d(Th)(u*v/dt + grad(u)'*grad(v))
                    -int2d(Th)(f*v*exp(-t)+uold*v/dt)
                    +on(1,u=0)+on(2,u=g*exp(-t)); // on résout le problème variationnel
    ff<<u(R+(Lx-R)/2,R+(Lx-R)/2)<<endl;
}
plot(u,wait=true,fill=true,ps="sol_quart.eps");
uex=(x-x^3)*(y-y^3)*exp(-T);
Vh err = abs(uex-u);
real L2err=sqrt( int2d(Th)((u-uex)^2)/ int2d(Th)(uex^2));
cout << "Erreur L2 relative (%)= " << 100*L2err << endl;
plot(uex,wait=true);
plot(err,wait=true);
```

We have depicted in Fig. 6 the solution at the final simulation time in $\widetilde{\Omega}$ and in $\widehat{\Omega}$ for comparison purposes. One can see that the two solutions match well.

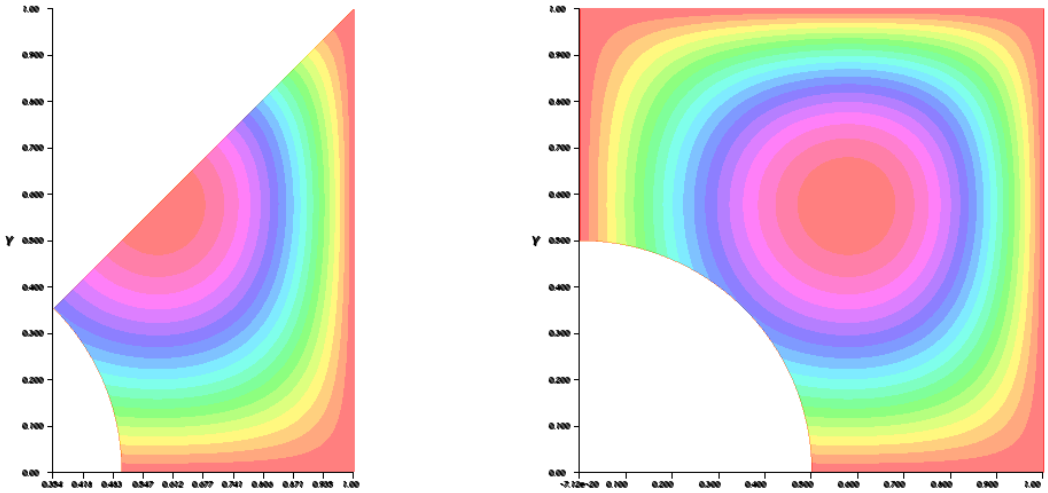


FIGURE 6 – Temperature distribution in the one eighth of the initial domain Ω at time $t = 1$.

5 a) We denote by Ω the computational cell depicted in figure 3 and by Γ_1, Γ_2, Γ_3, Γ_{4+}, Γ_{4-} and Γ_5 the various parts of its boundary as shown in figure 7.

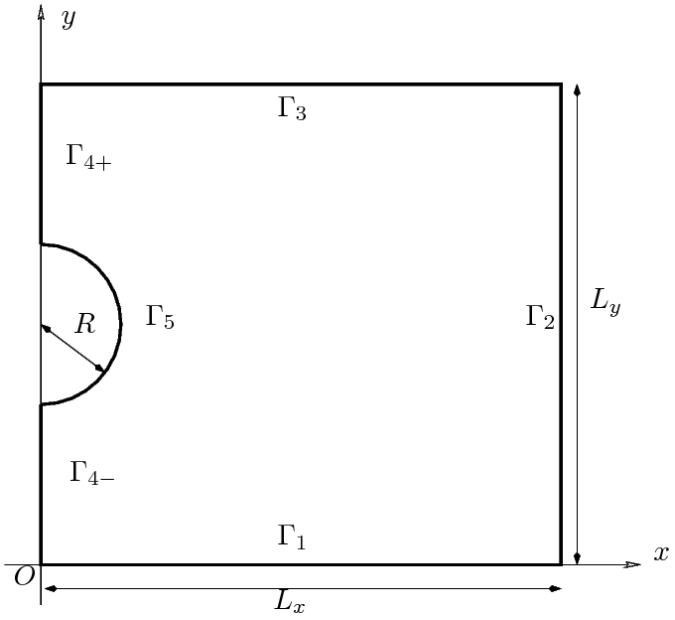


FIGURE 7 – Notations used.

The PDE problem to be solved reads : find $T(x,y,t)$ such that

$$\left\{\begin{array}{rcll} \rho\, c_p \frac{\partial T}{\partial t}(x,y,t) - k\,\Delta T(x,y,t) &=& 0 & \forall (x,y)\in\Omega \quad \forall t\in]0,t_{\max}] \\ -k\dfrac{\partial T}{\partial \nu}(x,y,t) &=& h_c\,(T(x,y,t)-T_e) & \forall (x,y)\in\Gamma_1\cup\Gamma_3 \quad \forall t\in]0,t_{\max}] \\ \dfrac{\partial T}{\partial n}(x,y,t) &=& 0 & \forall (x,y)\in\Gamma_2\cup\Gamma_{4-}\cup\Gamma_{4+} \quad \forall t\in]0,t_{\max}] \\ T(x,y,t) &=& T_0 & \forall (x,y)\in\Gamma_5 \quad \forall t\in]0,t_{\max}] \end{array}\right. \tag{14}$$

where $T_0 = 50 + 273$ K.
We introduce a uniform subdivision $\sigma = (t_n)_{n\in\{0,\ldots,N\}}$ of the time interval $[0,t_{\max}]$ with time step denoted δt and consider the following sequence of PDE problems deduced from (14) and indexed by $n\in\{1,\ldots,N\}$: find $T_n : (x,y)\in\Omega \mapsto T_n(x,y)\in\mathbb{R}$ such that

$$\left\{\begin{array}{rcll} \frac{\rho\, c_p}{\delta t} T_n(x,y) - k\,\Delta T_n(x,y) &=& \frac{\rho\, c_p}{\delta t} T_{n-1}(x,y) & \forall (x,y)\in\Omega \\ -k\dfrac{\partial T_n}{\partial \nu}(x,y) &=& h_c\,(T_n(x,y)-T_e) & \forall (x,y)\in\Gamma_1\cup\Gamma_3 \\ \dfrac{\partial T_n}{\partial \nu}(x,y) &=& 0 & \forall (x,y)\in\Gamma_2\cup\Gamma_{4-}\cup\Gamma_{4+} \\ T_n(x,y) &=& T_0 & \forall (x,y)\in\Gamma_5 \end{array}\right. . \tag{15}$$

The function T_n represents a first order accurate approximation of the temperature T in Ω at time t_n.
The weak formulation of problem (15) reads : find $T\in\mathbb{H}^1(\Omega)$ with $T = T_0$ on Γ_5 such that for all $v\in\mathbb{H}^1_{0,\Gamma_5}(\Omega)$

$$\frac{\rho\, c_p}{\delta t}\int_\Omega T_n\, v\ \mathrm{d}x\,\mathrm{d}y + k\int_\Omega \nabla T_n\cdot\nabla v\ \mathrm{d}x\,\mathrm{d}y + h_c\oint_{\Gamma_1\cup\Gamma_3} T_n\ v\ \mathrm{d}\gamma = \frac{\rho\, c_p}{\delta t}\int_\Omega T_{n-1}\, v\ \mathrm{d}x\,\mathrm{d}y + h_c\oint_{\Gamma_1\cup\Gamma_3} T_e\ v\ \mathrm{d}\gamma.$$

b) As a first step in the FREEFEM++ program, we need to define the parametrization of the boundary parts of the computational cell.

— A parametrization of the bottom border Γ_1 is $\gamma_1 : s\in[0,1]\longmapsto (s\,L_x, 0)$.

— A parametrization of the right edge border Γ_2 is $\gamma_2 : s \in [0,1] \longmapsto (s\,L_y, 0)$.
— A parametrization of the top border Γ_3 is $\gamma_3 : s \in [0,1] \longmapsto (s\,L_x, L_y)$.

We decompose the left edge border in 3 parts :

— a parametrization of the bottom line Γ_{4-} is $\gamma_{4-} : s \in [0,\alpha] \longmapsto (0, L_y\,s)$;
— a parametrization of the top line Γ_{4+} is $\gamma_{4+} : s \in [1-\alpha, 1] \longmapsto (0, L_y\,s)$;
— a parametrization of half circle Γ_5 is $\gamma_5 : s \in [\pi/2, \pi/2] \longmapsto (R\cos(s), (R+d)\sin(s))$;

where we have set

$$\alpha = \frac{1}{2}\left(1 - 2\frac{R}{L_y}\right) \quad \text{and} \quad d = \alpha\, L_y.$$

The following FREEFEM++ program solves the problem.

```
string filename; // filename of the file used to record the pictures
// ----  Mesh generation ----
real R=0.02;
real Lx=0.2, Ly=0.2;
real alpha=0.5*(1-2*R/Ly);
border Gamma1(t=0,1){x=Lx*t;y=0;label=1;};
border Gamma2(t=0,1){x=Lx;y=Ly*t;label=2;};
border Gamma3(t=0,1){x=Lx*t;y=Ly;label=3;};
border Gamma4p(t=1-alpha,1){x=0;y=Ly*t;label=2;};
border Gamma4m(t=0,alpha){x=0;y=Ly*t;label=2;};
border Gamma5(t=-pi/2,pi/2){x=R*cos(t);y=(R+alpha*Ly)+R*sin(t);label=4;};

int n=20;
mesh Th=buildmesh(Gamma1(n)+ Gamma2(n)+ Gamma3(-n)+Gamma4p(-n/2)+
                  Gamma5(-n/2)+Gamma4m(-n/2));
plot(Th,wait=true,ps="maillage_sol.eps");

// ---- Thermic data ----
real k=1.75; // Thermal conductivity of concrete [W m^{-1}K^{-1}]
real h=20; // Heat convection coefficient of concrete  [W m^{-2}K^{-1}]
real rho=2400; // Density of normal concrete in [kg m^{-3}]
real cp=750; // specific heat capacity in [J kg^{-1} K ^{-1}]
real Te1=5+273; // Upper room temperature in K
real Te3=20+273;// Lower room temperature in K
real T5=50+273; // Water temperataure
real Tini=15+273;// initial temperature of the concrete
// ---- FEM resolution ----
func u0=Tini; // initial data
int N=500;  // number of discretization nodes in the time domain
real tmax=5000, dt=tmax/N; // Time windows length in sec. and time-step

fespace Vh(Th,P2);
Vh u,v,uold,T;
macro grad(u) [dx(u),dy(u)] //fin macro
u=u0; // initialisation
problem  chaleur(u,v)=  int2d(Th)(rho*cp*u*v/dt) + int2d(Th)(k*grad(u)'*grad(v))
       - int2d(Th)(rho*cp*uold*v/dt) + int1d(Th,1)(h*u*v) + int1d(Th,3)(h*u*v)
        - int1d(Th,1)(h*Te1*v) - int1d(Th,3)(h*Te3*v)
        + on(4,u=T5);
for (real t=0;t<tmax;t+=dt){// loop over the time discretization nodes
uold=u;
chaleur;
// we record the solution from time to time
if (((t/dt)%50) == 0)
{
T=u-273; // Temperature in Celsius degree
```

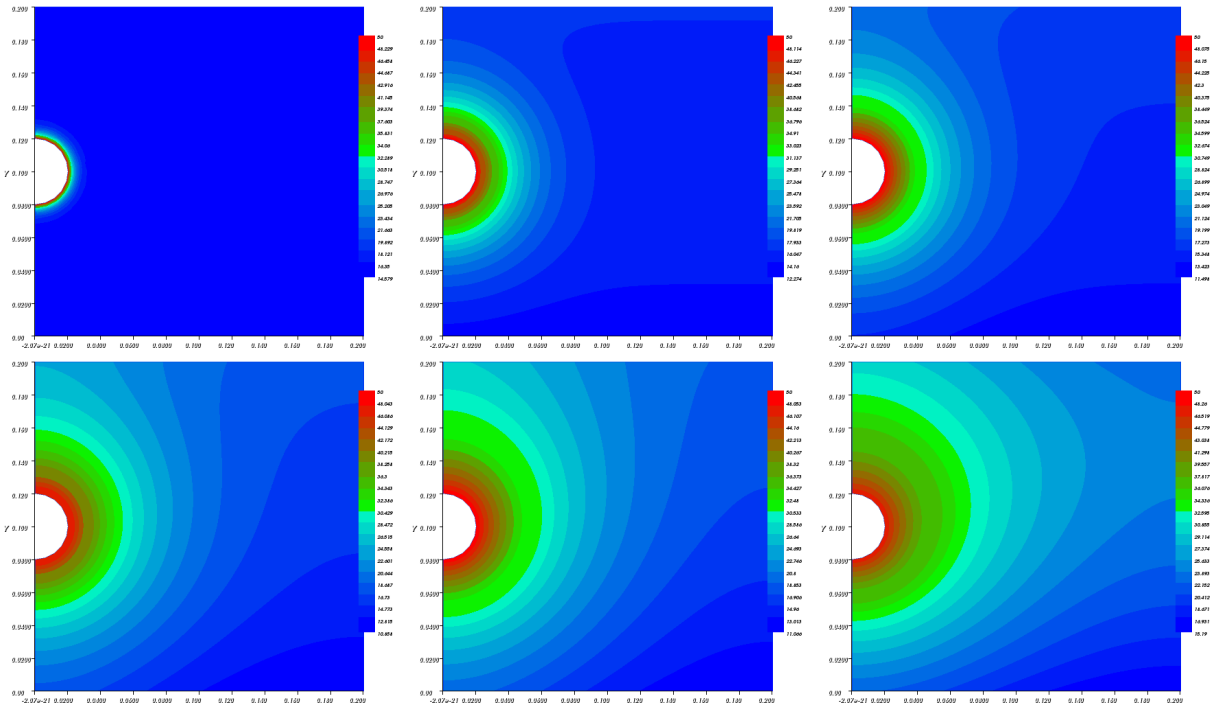


FIGURE 8 – Temperature in the concrete slab at time $t = 0, 750, 1500$ sec. on the top line and at time $t = 3000, 5000$ sec. as well as steady state solution on the bottom line.

```
filename="temperature"+t+".eps";
     plot(T,fill=true,ps=filename);
}
}
T=u-273; // Temperature in Celsius degree
filename="temperature"+tmax+".eps";
plot(T,fill=true,ps=filename);
```

The temperature at various time is depicted in Fig. 8.

c) The steady state regime is obtained when the temperature in the concrete slab doesn't change anymore with time. Thus from (14), we deduce that the temperature at the steady state regime satisfies

$$\left\{\begin{array}{rcll} -k\,\Delta T(x,y) & = & 0 & \forall (x,y)\in\Omega \\ -k\dfrac{\partial T}{\partial \nu}(x,y) & = & h_c\,(T(x,y)-T_e) & \forall (x,y)\in\Gamma_1\cup\Gamma_3 \\ \dfrac{\partial T}{\partial n}(x,y) & = & 0 & \forall (x,y)\in\Gamma_2\cup\Gamma_{4-}\cup\Gamma_{4+} \\ T(x,y) & = & T_0 & \forall (x,y)\in\Gamma_5 \end{array}\right. \tag{16}$$

The weak formulation of problem (16) reads : find $T\in\mathbb{H}^1(\Omega)$ with $T=T_0$ on Γ_5 such that for all $v\in\mathbb{H}^1_{0,\Gamma_5}(\Omega)$

$$k\int_\Omega \nabla T_n\cdot\nabla v\ \mathrm{d}x\,\mathrm{d}y + h_c\oint_{\Gamma_1\cup\Gamma_3} T_n\ v\ \mathrm{d}\gamma = h_c\oint_{\Gamma_1\cup\Gamma_3} T_e\ v\ \mathrm{d}\gamma.$$

The following FREEFEM++ program computes the temperature at the steady state regime.

```
// ----  Mesh generation ----
real R=0.02;
real Lx=0.2, Ly=0.2;
real alpha=0.5*(1-2*R/Ly);
border Gamma1(t=0,1){x=Lx*t;y=0;label=1;};
border Gamma2(t=0,1){x=Lx;y=Ly*t;label=2;};
border Gamma3(t=0,1){x=Lx*t;y=Ly;label=3;};
border Gamma4p(t=1-alpha,1){x=0;y=Ly*t;label=2;};
border Gamma4m(t=0,alpha){x=0;y=Ly*t;label=2;};
border Gamma5(t=-pi/2,pi/2){x=R*cos(t);y=(R+alpha*Ly)+R*sin(t);label=4;};

int n=20;
mesh Th=buildmesh(Gamma1(n)+ Gamma2(n)+ Gamma3(-n)+Gamma4p(-n/2)+
                  Gamma5(-n/2)+Gamma4m(-n/2));
plot(Th,wait=true,ps="maillage_sol.eps");

// ---- Thermic data ----
real k=1.75; // Thermal conductivity of concrete [W m^{-1}K^{-1}]
real h=20; // Heat convection coefficient of concrete  [W m^{-2}K^{-1}]
real Te1=5+273; // Upper room temperature in K
real Te3=20+273;// Lower room temperature in K
real T5=50+273; // Water temperataure

// ---- FEM resolution ----
fespace Vh(Th,P2);
Vh u,v;
macro grad(u) [dx(u),dy(u)] //fin macro

solve chaleur(u,v)=int2d(Th)(k*grad(u)'*grad(v))
                   + int1d(Th,1)(h*u*v) + int1d(Th,3)(h*u*v)
                   -  int1d(Th,1)(h*Te1*v) -int1d(Th,3)(h*Te3*v)
                   +on(4,u=T5);
Vh T=u-273; // Temperature in Celsius degree
plot(T,wait=true,fill=true);
```

6 a) The weak formulation for problem (8) reads : find $u_0 \in \mathbb{H}^1(\Omega)$ with $u_0 = T_b - T_a$ on Γ_1 such that for all $v \in \mathbb{H}^1_{0,\Gamma_0}(\Omega)$

$$ke_z \int_\Omega \nabla u_0(x) \cdot \nabla v(x)\,\mathrm{d}x + 2h_c \int_\Omega u_0(x)\, v(x)\,\mathrm{d}x = 0 \tag{17}$$

where we have denoted $\mathbb{H}^1_{0,\Gamma_0}(\Omega)$ the sub-space of $\mathbb{H}^1(\Omega)$ of functions with a trace on Γ_0 equal to zero.
The weak formulation for problem (9) reads : find $u_i \in \mathbb{H}^1(\Omega)$ with $u_i = T_b - T_a$ on Γ_1 such that for all $v \in \mathbb{H}^1_{0,\Gamma_0}(\Omega)$

$$ke_z \int_\Omega \nabla u_i(x) \cdot \nabla v(x)\,\mathrm{d}x + 2h_c \int_\Omega u_i(x)\, v(x)\,\mathrm{d}x + 2\epsilon\sigma \int_\Omega g(u_{i-1}) u_i(x)\, v(x)\,\mathrm{d}x = 0. \tag{18}$$

b) We have depicted in figure 9 the temperature distribution in the plate (left picture) and the temperature on a vertical cross section as a function of the y coordinate (right picture).

7 The following FREEFEM++ program solves the transient problem.

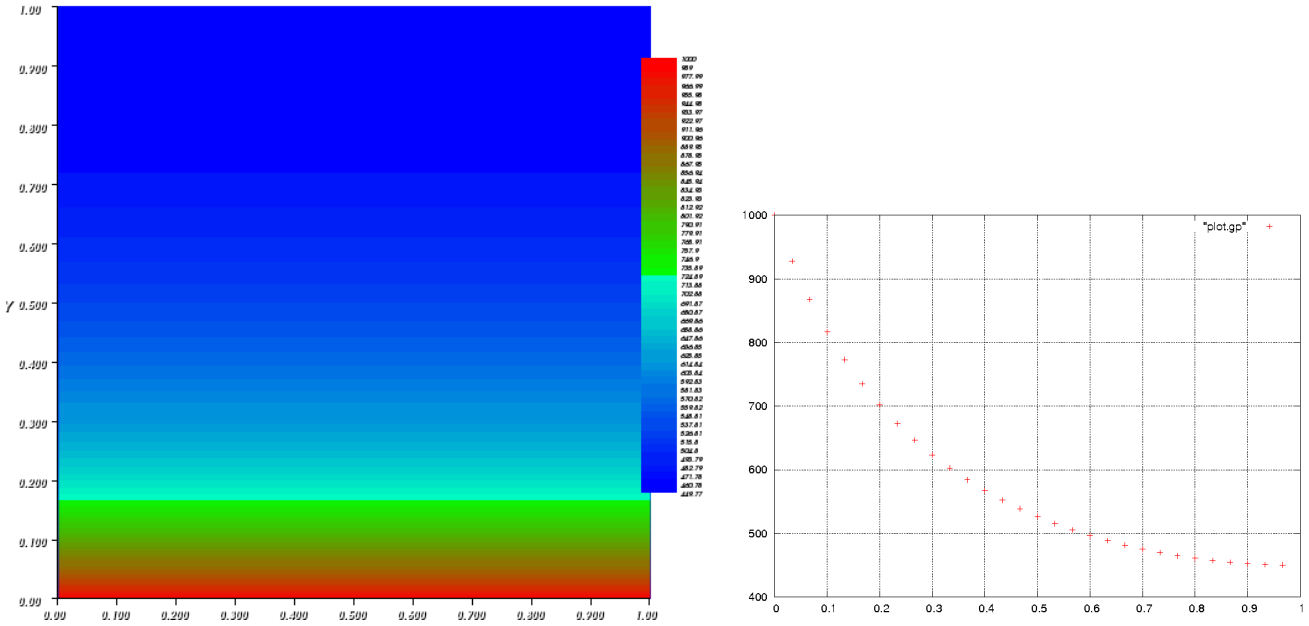


FIGURE 9 – Temperature inside the plate : bi-dimensional rendering on the left and temperature on a vertical cross section.

```
verbosity=1;
real L=1; // size of the plate in m
int Np=30; // nb of discretization nodes per square edge
mesh Th=square(Np,Np,[L*x,L*y]);
plot(Th,ps="main3_maillage.ps");

real k = 400; // thermal conductivity of copper, W/(m-K)
real rho = 8960; // density of copper, kg/m^3
real Cp = 386; // specific heat of copper, J/(kg-K)
real tz = .01; // plate thickness in meters
real stefanBoltz = 5.670373e-8; // Stefan-Boltzmann constant, W/(m^2-K^4)
real h = 1; // Convection coefficient, W/(m^2-K)
real Ta = 300; //  ambient temperature in degree-Kelvin
real Tb = 1000; // temperature at the bottom edge
real epsilon = .5; // emissivity of the plate surface

fespace Vh(Th,P2);
Vh u,uold,v,b=0;
macro grad(u)[dx(u),dy(u)]//end of macro

int Nt=400; //  number of discretization nodes in the time domain
real Tf=5000, dt=Tf/Nt; // Time windows length in sec. and time-step
u=300; // initial temperature in unit K

int iterx=200, i=0,n=0;
real err=1, normu=0;
ofstream ff("res_main3.txt");
// Loop over the time windows
for (real t=0;t<Tf;t+=dt){
    cout << "Progress = " << (100*n/Nt) << " % " << endl;
uold=u;
        b=0;
```

```
solve thermnl(u,v)=int2d(Th)( (rho*Cp*tz/dt)*u*v+
                                    k*tz*grad(u)'*grad(v)+2*h*u*v)
                     +int2d(Th)(b*u*v)
                     -int2d(Th)((rho*Cp*tz/dt)*uold*v)
+on(1,u=Tb-Ta);
iterx=200; i=0;
err=1; normu=0;
while (err>1E-6 & i<iterx){
i=i+1; // iteration counter
uold=u;
    b=2*stefanBoltz*epsilon*(uold+2*Ta)*((uold+Ta)^2+Ta^2);
     thermnl; // problem is solved
normu=sqrt(int2d(Th)(abs(u)^2)); // norm of the solution
     err=sqrt(int2d(Th)(abs(u-uold)^2))/normu; // difference with the last iterate
}
    ff<<u(L/2,L)<<endl; // solution at the top edge center
n=n+1;
}
//cout << "Number of iterations = " << i << endl;
Vh T=u+Ta; // Temperature in Kelvin

plot(T,ps="fig_plate_dynamic.eps",fill=true,nbiso=50);
cout << "Temperature at the top edge = " << T(L/2.,L) << endl;
normu=sqrt(int2d(Th)(abs(u+Ta)^2));
cout << "Norm of the solution = "<< normu << endl;
```

We have depicted in figure 10 the temperature inside the plate after 5000 sec. This figure has to be compared with the steday state solution depicted in figure 9.

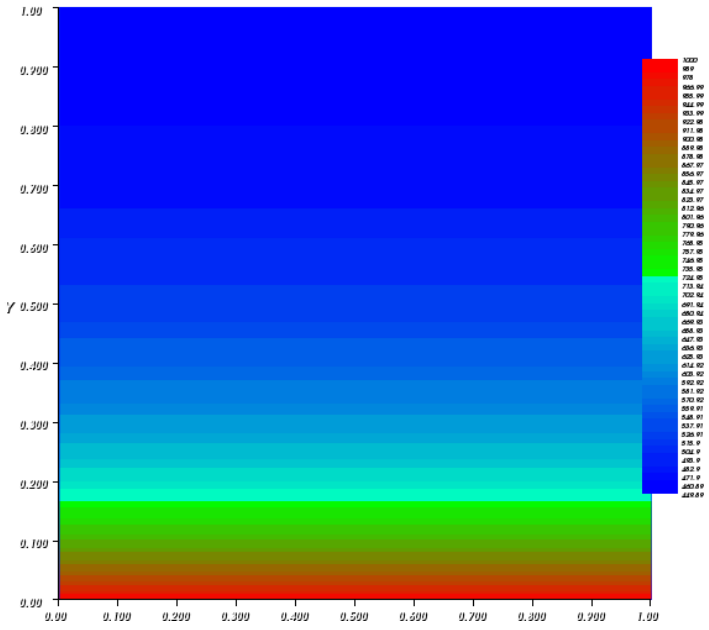


FIGURE 10 – Temperature inside the plate : bi-dimensional rendering after 5000 sec.

We have also depicted in figure 11 the temperature inside the plate after 5 sec., 50 sec. and 500 sec. in the transient regime. We can see that the temperature variations are low during the transient regime (the scale on the three pictures are slightly different). The temperature variations can be enhanced by changing the color scheme, as shown in figure 12 where the temperature inside the plate after 5 sec. and 5000 sec.

are compared.

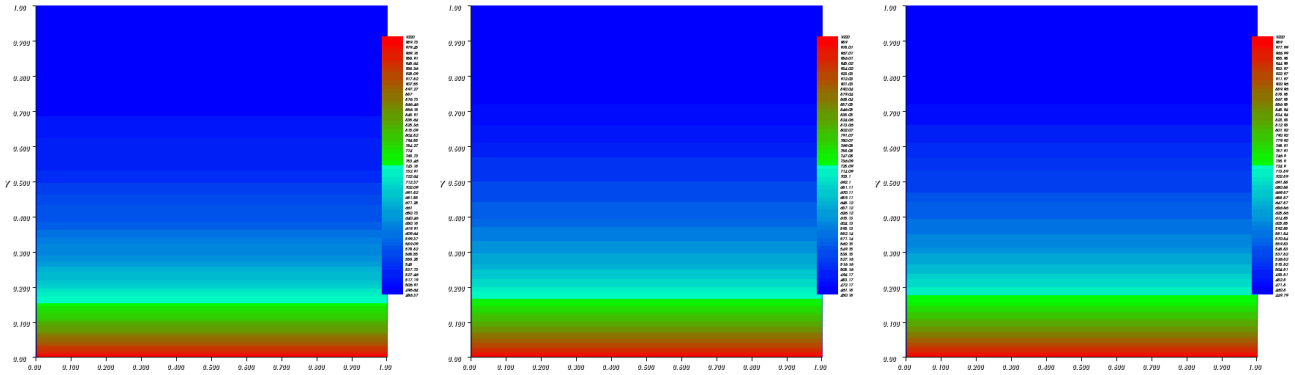


FIGURE 11 – Temperature inside the plate : bi-dimensional rendering after 5 sec., 50 sec. and 500 sec. with a RGB color scheme.

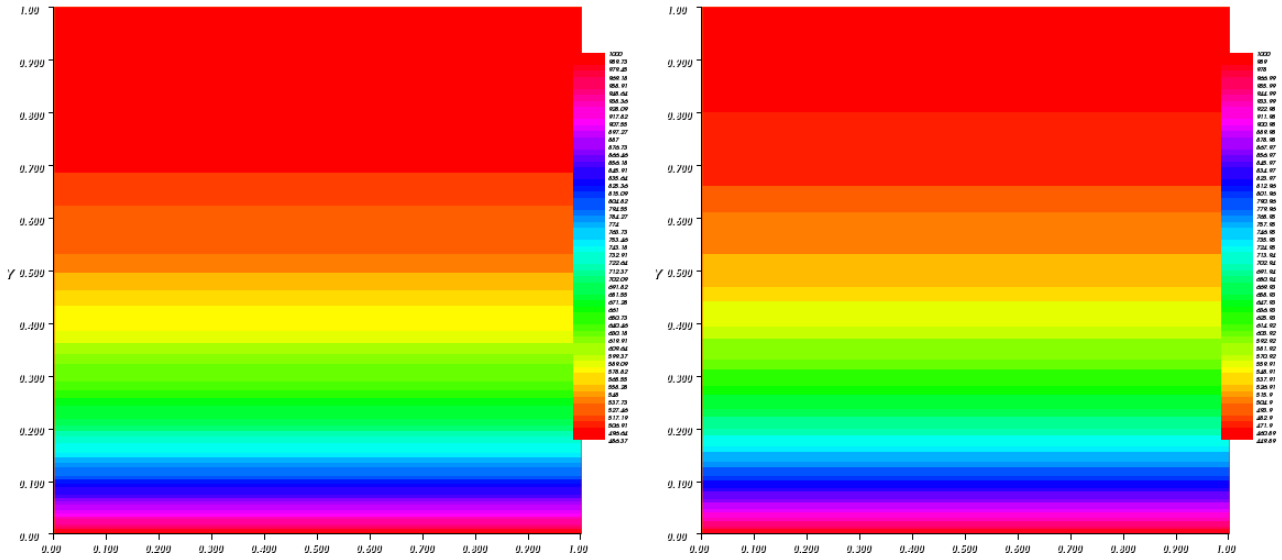


FIGURE 12 – Temperature inside the plate : bi-dimensional rendering after 5 sec., and 5000 sec. with a HSV color scheme.